POLIMI GRADUATE SCHOOL OF MANAGEMENT

ALGORITHMIC TRADING — HACKATHON PROJECT REPORT

Optimized Adaptive Trading Agent

A composite-signal agent trained via differential evolution on the ABIDES RMSC04 market simulator

Group 3 – FQ

Marco Caputo
Mario Giulio Carta
Nicolò Ferrari
Antonio Fontanella
Giovanni Mangone
Lorenzo Meloncelli
Lorenzo Padula
Luca Palumbo

POLIMI Graduate School of Management · Master in Quantitative Finance (MFQ)
Build and Battle Your Trading Agent · 28 May 2026

1. The ABIDES Framework

ABIDES (Agent-Based Interactive Discrete Event Simulation) is an open-source platform for market microstructure research introduced by Byrd et al. (2020). It provides a realistic, reproducible environment in which heterogeneous trading agents interact through a simulated limit order book, replicating the dynamics of a liquid equity market without deploying real capital. The simulator is driven by a discrete-event kernel that maintains a priority queue of triplets $(\, t,\, \text{agent}_i,\, \text{message}\,)$ and jumps from event to event rather than advancing in fixed steps, making it efficient over full trading-day horizons measured in nanoseconds. Communication between agents is strictly asynchronous and routed through the kernel, mirroring the information asymmetry of real markets.

All experiments used the **RMSC04** configuration (Reference Market Simulation Configuration, v4), calibrated to reproduce a realistic US equity trading day on ticker `ABM`. The background population consists of 1 `ExchangeAgent`, 2 adaptive POV `MarketMakerAgent`s, 102 `ValueAgent`s, 12 `MomentumAgent`s, and 1 000 `NoiseAgent`s — a mix that produces a mid-price tracking the underlying fundamental and an order book with realistic spread, depth, and volume profiles.

2. Agent Design and Strategy

2.1 Architecture

Our agent, `OptimizedAgent`, is a `TradingAgent` subclass that wakes up every 60 seconds. On each wakeup it queries the best bid and ask, updates an internal price history, and evaluates a composite signal before deciding whether to submit a limit order or stay flat. Combining three complementary signals — rather than relying on a single factor — reduces false positives, since each signal captures a different dimension of market dynamics.

At bootstrap the agent loads pre-calibrated parameters from `best_params.npy`, falling back to hard-coded defaults if the file is absent. This separation between training and deployment means the same class can be used both inside the optimizer and in the live tournament without modification.

2.2 Signal Construction

Three normalized signals are computed at each tick from the latest bid-ask midpoint:

- **Mean-Reversion Signal (MRS).** Measures the deviation of the midpoint from its N-period rolling mean in standard deviation units. A price below the mean produces a positive signal, encouraging a buy; a price above it produces a negative signal, favouring a sale:

$$\text{MRS} = \frac{\bar{p} - p_t}{\sigma}$$

- **Momentum Signal (MS).** Compares a fast EMA (span 5) with a slow EMA (span 20). A positive gap indicates an uptrend; a negative gap a downtrend:

$$\text{MS} = \frac{\text{EMA}_{\text{fast}} - \text{EMA}_{\text{slow}}}{\text{EMA}_{\text{slow}}/1000}$$

- **Inventory Signal (IS).** Penalizes directional exposure to keep the book balanced over the day:

$$\text{IS} = -\frac{\text{position}}{\text{max\_position}}$$

The three signals are combined into a single composite score:

$$\text{composite} = w_1 \cdot \text{MRS} + w_2 \cdot \text{MS} + w_3 \cdot \text{IS}$$

If `composite` exceeds a threshold τ the agent buys at the ask; if it falls below $-\tau$ it sells at the bid; otherwise it holds. Sizes are capped by `max_position`.

3. Parameter Optimization

The seven parameters (`window`, w_1, w_2, w_3, τ, `order_size`, `max_position`) were tuned offline via `scipy.optimize.differential_evolution` (DE), a population-based global optimizer well-suited to noisy, non-convex objectives.

Differential Evolution is a population-based stochastic optimizer introduced by Storn & Price (1997). It maintains a population of NP candidate vectors $\{\boldsymbol{\theta}_i\}$; at each generation, every vector is challenged by a trial vector constructed via mutation and crossover. Mutation combines three randomly selected population members: $\mathbf{v} = \boldsymbol{\theta}_a + F(\boldsymbol{\theta}_b - \boldsymbol{\theta}_c)$, where $F \in (0, 2]$ is a scale factor. The trial vector replaces the target only if it achieves a lower objective value, so the population monotonically improves. DE requires no gradient information and handles non-smooth, multimodal landscapes — properties well-suited to a simulator-based objective that is noisy and non-differentiable.

The fitness function evaluates any candidate vector $\boldsymbol{\theta}$ by running it across five independent RMSC04 seeds $\{42, 123, 7, 999, 2024\}$ and returning the negative average mark-to-market PnL. Averaging over seeds prevents the optimizer from fitting to a lucky market realization and gives a more reliable estimate of expected performance. Each evaluation uses a half-day window (up to 13:00) to keep computation time manageable.

The DE setup uses a population multiplier of 5, a maximum of 30 iterations, and full CPU parallelism via Python multiprocessing. An early-stopping callback halts the search after seven consecutive generations without meaningful improvement. A final L-BFGS-B polishing step refines the best candidate to a continuous local optimum, and the result is saved to `best_params.npy`.

Optimization converged to the parameter vector reported in Table 1. The solution is dominated by the mean-reversion weight ($w_1 = 2.5$), with momentum essentially switched off ($w_2 \approx 0$) and a moderate inventory penalty ($w_3 = 1.0$). This is consistent with the structure of RMSC04: prices are anchored to an OU-like fundamental, so contrarian signals carry more predictive content than short-horizon trend-following ones. Notably, the optimized `order_size` (30 shares) is well below the upper bound of 150, and `max_position` (550) sits near the top of its range, reflecting a preference for frequent small trades over large concentrated bets.

Parameter	**Optimized value**	**Search bounds**	**Interpretation**
`window`	30	[5, 60]	Rolling mean/std window (ticks)
w_1	2.5	[0, 3]	Mean-reversion weight
w_2	0	[0, 3]	Momentum weight (switched off)
w_3	1.0	[0, 3]	Inventory penalty weight
τ	1.5	[0.05, 2.0]	Decision threshold
`order_size`	30	[10, 150]	Shares per order
`max_position`	550	[100, 600]	Maximum net inventory

Table 1: Optimized parameter vector from differential evolution, with search bounds and interpretation.

4. Results and Discussion

The agent was evaluated both in the optimization loop and during the live hackathon tournament. In-sample, the tuned parameters generated positive average PnL across all five training seeds, confirming convergence to a genuinely profitable policy rather than overfitting to a single realization.

In simulation, the agent places frequent but modest-sized limit orders that keep net inventory close to zero. The inventory signal plays a key stabilizing role: even when the mean-reversion signal is strongly positive, a large accumulated position tempers the composite score and prevents the agent from compounding a directional bet at unfavorable prices.

The main limitation is that optimization was conducted on a half-day window, a trade-off between speed and accuracy that may leave the agent slightly mis-calibrated for the second half of the session, when intraday liquidity and volatility patterns differ from the open. Future work should re-run optimization on full-day simulations and explore a time-of-day feature that scales signal sensitivity with intraday volatility. A further direction would be to replace the fixed linear combination with a data-driven weighting scheme — such as a logistic regression trained on historical fills — though careful handling of look-ahead bias would be required.

5. Conclusions

This project built and evaluated a complete pipeline for algorithmic strategy development within ABIDES: from a minimal baseline agent, through composite signal design, to offline parameter tuning via differential evolution. The RMSC04 configuration proved to be a credible proxy for a liquid equity market, and the optimized agent successfully exploits the mean-reverting structure of the simulated price process while keeping inventory risk under control. The formalized calibration routine provides a reproducible, systematic alternative to manual parameter selection. The main directions for future work are full-day optimization and adaptive, data-driven signal weighting.

References

[1] Byrd, D., Hybinette, M., & Balch, T. (2020). *ABIDES: Towards high-fidelity market simulation for AI research.* ACM International Conference on AI in Finance (ICAIF).

[2] Storn, R., & Price, K. (1997). Differential Evolution — A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341–359.